

3 Assessments & Grading

3.1 Grading Scheme

Component	Weight
Quizzes	15%
Midterm exam	15%
Final exam	25%
Homeworks	25%
Project	20%
Total	100%

3.2 Final Project

The goal of this project is to design and implement a complete computer graphics application that integrates concepts from rasterization and, where applicable, ray tracing. The project should demonstrate a solid understanding of core techniques from the course as well as additional advanced techniques explored independently.

You may choose one of the following directions or propose your own idea.

- *Scene Rendering System*: Develop a visually rich 3D scene with multiple objects, lighting, and interactive camera control. Possible extensions include shadow mapping, environment mapping, or physically based rendering.
- *Mini Game (Graphics-Focused)*: Create a small interactive game where graphics plays a central role. This may include dynamic lighting, shader-based effects, or simple physics. More advanced features may include particle systems, animation, or post-processing effects.
- *Simulation or Visualization*: Design a system that visualizes a physical simulation or procedural terrain. Possible extensions include GPU-based methods, level of detail, or real-time interaction.
- *Rendering Techniques Showcase*: Implement and compare multiple rendering approaches, such as different shading models or rendering pipelines. You may include performance comparisons and custom shader variations.
- *Experimental or Creative Graphics*: Explore artistic or unconventional ideas such as non-photorealistic rendering, procedural art, or audio-reactive visuals.

Projects should go beyond basic functionality and demonstrate technical depth, visual quality, and thoughtful design.

Technical Expectations

Baseline Requirements: All projects are expected to implement the fundamental techniques covered in lectures, adapted as appropriate to the chosen direction. These include the Phong reflection model with ambient, diffuse, and specular components, along with appropriate material properties such as shininess and reflectance coefficients. Lighting should preferably be computed per-fragment for improved visual quality.

Projects should also support texture mapping, including diffuse maps and specular maps with proper sampling in the fragment shader. The system should incorporate multiple light types: directional lights, point lights with distance attenuation, and spotlights with smooth falloff using inner and outer cutoff angles. Correct accumulation of contributions from multiple lights is required.

From an implementation standpoint, projects should demonstrate proper use of GLSL shaders, including uniform management for lights and materials, and correct handling of transformations and surface normals.

Note on applicability: Not every baseline technique will fit every project direction. If a technique does not apply to your chosen direction, state this clearly in your proposal and confirm with the instructor before proceeding.

Advanced Techniques: In addition to the baseline requirements, each project must implement at least two advanced techniques from the list below. The combined effort score of the chosen techniques must be at least **6 out of 10**. Note that selecting two low-effort techniques (e.g., Fog + Gamma Correction, combined score 4) does not meet this requirement; at least one technique must have an effort score of 3 or higher.

Shadow Mapping (Effort: 4/5) Renders the scene from the light's point of view to generate a depth map, which is then used during rendering to determine shadowed regions. Requires handling light-space transformations, depth comparisons, and resolving artifacts such as shadow acne.

Normal Mapping (Effort: 3/5) Enhances surface detail without increasing geometric complexity by perturbing surface normals using a texture. Requires working in tangent space and constructing a TBN (tangent, bitangent, normal) basis.

Parallax Mapping (Effort: 4/5) Simulates depth within textures by adjusting texture coordinates based on the view direction and a height map. Provides a stronger illusion of depth than normal mapping but is more complex to implement.

Environment Mapping (Effort: 3/5) Simulates reflections by sampling from a cube map representing the surrounding environment. Requires computing reflection vectors in the fragment shader.

Refraction (Effort: 3/5) Simulates the bending of light through transparent materials such as glass. Involves computing refraction vectors and applying an index of refraction to control the effect.

Fog (Effort: 2/5) Enhances depth perception by blending object colors with a fog color based on camera distance. Linear or exponential fog models may be used.

Bloom (Effort: 4/5) A post-processing effect that creates a glow around bright areas of the image. Typically requires extracting bright regions, applying a blur, and combining the result with the original image using framebuffer operations.

Gamma Correction (Effort: 2/5) Ensures that rendered colors appear correctly on display devices by converting from linear color space to gamma space.

Screen Space Ambient Occlusion (Effort: 5/5) Approximates ambient occlusion in screen space by sampling nearby geometry using depth and normal information. Significantly improves visual realism but is computationally and conceptually challenging.

Physically-Based Rendering (Effort: 4/5) Replaces empirical lighting models with physically motivated formulations using parameters such as metallic and roughness values. Produces more realistic and consistent results across different lighting conditions.

Deferred Shading (Effort: 5/5) Separates geometry processing and lighting into distinct passes by storing intermediate data (position, normal, color) in a G-buffer. Enables efficient rendering with many light sources but requires significant architectural changes to the rendering pipeline. Due to its scope, this technique is best suited as the central focus of the project rather than a secondary addition.

Toon Shading (Effort: 3/5) Produces a stylized, cartoon-like appearance by quantizing lighting values into discrete levels. May be combined with edge detection for a stronger visual effect.

Summary of Requirements:

- Implement the baseline techniques covered in lectures, adapted as appropriate to the chosen direction.
- Implement at least two advanced techniques with a combined effort score of at least 6/10.
- At least one chosen technique must have an effort score of 3 or higher.
- Implementations demonstrating higher complexity or additional advanced features may be considered for bonus credit (up to 5% of the final project grade).

Assets and Resources

You may use external models and textures provided they are free to use and properly credited in your report. Suggested sources include:

- <https://www.blendswap.com/> — A community-driven library of free 3D models created with Blender. Assets are shared under Creative Commons licenses.
- <https://3dwarehouse.sketchup.com/> — A large public repository of user-contributed 3D models spanning architecture, furniture, vehicles, and more. Models are primarily in SketchUp format but can be exported to other formats.
- <https://polyhaven.com/> — A high-quality library of HDRIs, PBR texture maps (diffuse, roughness, normal, displacement), and 3D models. All assets are released under the CC0 license, meaning they may be used freely for any purpose without attribution. Particularly useful for realistic lighting and material work.

The emphasis should remain on implementing graphics techniques rather than collecting assets.

Milestones and Evaluation

The project will be evaluated based on technical complexity, correctness of implementation, code quality, visual output, and presentation. Greater weight will be given to technical depth and correctness.

Milestone	Weight
Proposal	2%
Progress Report	3%
Final Submission & Presentation	15%

Proposal should describe the chosen project direction, the techniques you plan to implement (baseline and advanced), and any external assets or resources you intend to use. It should also include a brief implementation plan.

Progress Report should summarize what has been implemented so far, what is working, and what remains. Include screenshots or renders if available.

Final Submission should include a runnable implementation, complete source code, and a short written report describing the project, the techniques implemented, and challenges encountered. A live demonstration will be conducted as part of the final presentation.

Guidelines

Begin with a simple working version and incrementally add features. Focus on building a polished system around a few strong ideas rather than leaving many features incomplete. Be prepared to explain all implemented techniques during the final presentation, including any learned independently.